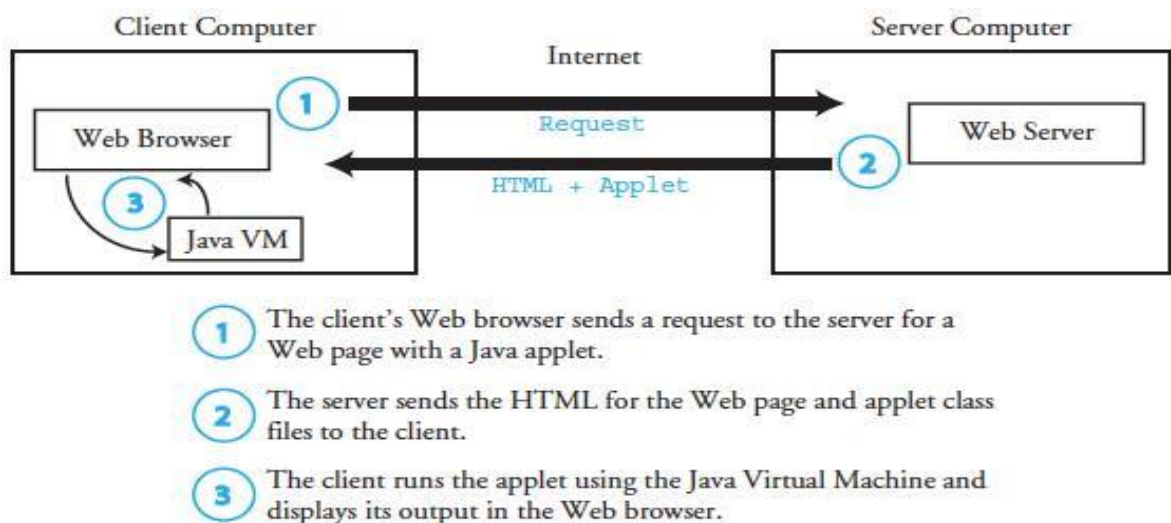


Unit:10**JAVA SERVER PAGE (JSP/SERVLET TECHNOLOGY) [7Hrs]****10.1 JSP/Servlet Technology overview:**

- The web application is built with java that includes Java applets, Java server pages and Java Servlet.
- The applet, JSP and servlet technology are used for web development and to run the web applications smoothly and actively.
- The applet runs on the client machine whereas the JSP and servlet runs on the server of the machine.
- A Java Server Pages component is a type of Java servlet that is designed to fulfill the role of a user interface for a Java web application.

**Fig: Running a Java Applet****10.2 Servlet Life Cycle, Creating and Deploying New Servlet:****10.2.1. Servlet:**

- Java Servlets are programs that run on a Web or Application server and act as a middle layer between a request coming from a Web browser or other HTTP client and databases or applications on the HTTP server.
- Servlet is a web component that is deployed on the server to create a dynamic web page.

10.2.2 Advantages of Servlet:

- **Better performance:** because it creates a thread for each request, not process.
- **Portability:** because it uses Java language.
- **Robust:** JVM manages Servlets, so we don't need to worry about the memory leak, garbage collection, etc.
- **Secure:** because it uses java language
- **Full Functionality:** The Java class libraries is available to a servlet which makes it full functionable.
- **Fast:** Since servlets are compiled into bytecodes, they can execute more quickly as compared to other scripting languages.

10.2.3. Servlet Life cycle:

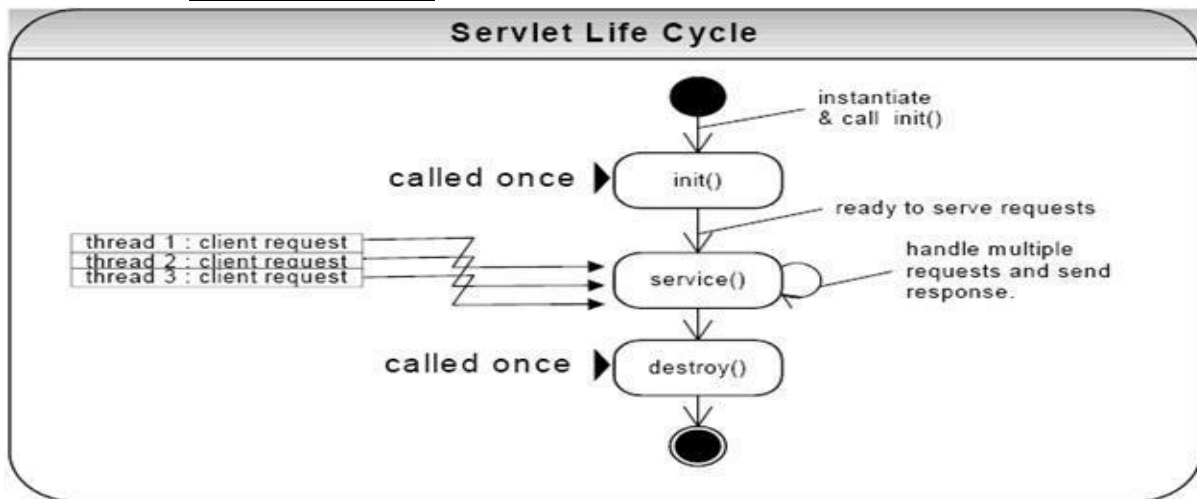


Fig: Life cycle of java Servlet

The life cycle of servlet contains the three methods. They are `init()`, `service()` and `destroy()`

- Servlet life cycle contains five steps:
 - 1) Loading of Servlet
 - 2) Creating instance of Servlet
 - 3) Invoke `init()` once
 - 4) Invoke `service()` repeatedly for each client request
 - 5) Invoke `destroy()`

Step 1: Loading of Servlet:

- ✓ When the web server (e.g. Apache Tomcat) starts up, the servlet container deploys and loads all the servlets.
- ✓ The web browser generates an HTTP request for the entered URL then, this request sent to appropriate sever.

Step 2: Creating instance of Servlet:

- ✓ Once all the Servlet classes loaded, the servlet container creates instances of each servlet class.
- ✓ The request loaded in the servlet is mapped to the particular servlet and it dynamically retrieved and loaded into address space of particular server.

Step 3: Invoke `init()` method:

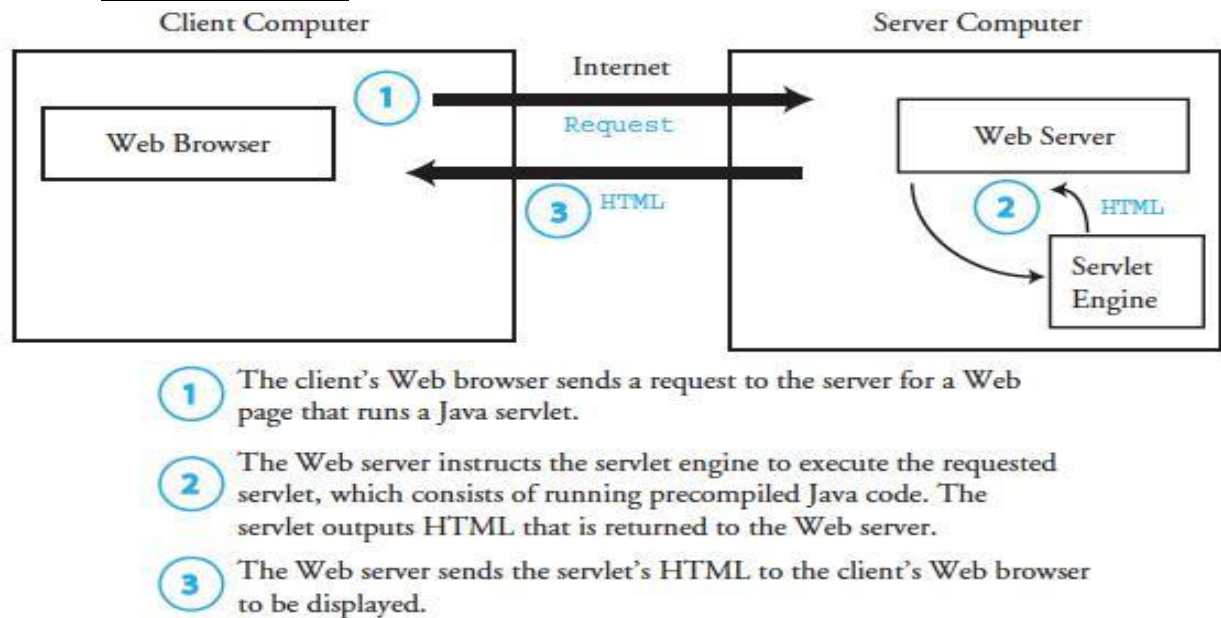
- ✓ Once all the servlet classes are instantiated, the `init()` method is invoked for each instantiated servlet.
- ✓ This method initializes the servlet.
- ✓ This method is invoked when the servlet is first loaded into the memory.

Step 4: Invoke `service()` method:

- ✓ Each time the web server receives a request for servlet, it invokes `service()` method.
- ✓ If the servlet is `GenericServlet` then the request is served by the `service()` method itself, if the servlet is `HttpServlet` then `service()` method receives the request and dispatches it to the correct handler method based on the type of request.
- ✓ The `service()` method can be called any number of times during servlet life cycle.
- ✓ Out of all the 5 steps in life cycle, this is the only step that executes multiple times.

Step 5: Invoke destroy() method:

- ✓ when the web server is stopped, it unloads all the servlets and calls destroy() method for each initialized servlets.
- ✓ This method relinquishes the resources that are allocated for the servlet.
- ✓ The garbage collection is done for the memory allocated to servlet and its objects.

10.2.4. Running a Servlet:**Fig: Running a Java Servlet****Simple Example of servlet:**

```
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;
public class DemoServlet extends HttpServlet
{
    public void doGet(HttpServletRequest req,HttpServletResponse res)
    throws ServletException,IOException
    {
        res.setContentType("text/html");//setting the content type
        PrintWriter pw=res.getWriter();//get the stream to write the data

        //writing html in the stream
        pw.println("<html><body>");
        pw.println("Welcome to servlet");
        pw.println("</body></html>");

        pw.close();//closing the stream
    }
}
```

10.2.5. The Servlet API:

- Two packages contain the classes and interfaces that are required to build servlets. These are ***javax.servlet*** and ***javax.servlet.http***.
- They constitute the Servlet API.

A) The *javax.servlet* Package:

- ✓ The ***javax.servlet*** package contains a number of interfaces and classes that establish the framework in which servlets operate.

The following table summarizes the **core interfaces** that are provided in this package.

Interface	Description
Servlet	Declares life cycle methods for a servlet.
ServletConfig	Allows servlets to get initialization parameters.
ServletContext	Enables servlets to log events and access information about their environment.
ServletRequest	Used to read data from a client request.
ServletResponse	Used to write data to a client response.

The **core classes** that are provided in the ***javax.servlet*** package are shown in the table below:

Class	Description
GenericServlet	Implements the Servlet and ServletConfig interfaces.
ServletInputStream	Provides an input stream for reading requests from a client.
ServletOutputStream	Provides an output stream for writing responses to a client.
ServletException	Indicates a servlet error occurred.
UnavailableException	Indicates a servlet is unavailable.

B) The *javax.servlet.http* Package:

- ✓ This contains no of interfaces and classes which are commonly used by servlet developers.
- ✓ The interfaces provided in the packages are:

Following table shows the interfaces of this packages:

Interface	Description
Http servlet Request	Enable servlet to read data from HTTP request
Http Servlet Response	Enable servlet to read data from HTTP request
Http Session	Allows session data to read and write
Http Session Binding Listener	Informs an object that is bound to or unbound from a session

Following table shows the classes of this packages:

Class	Description
Cookie	Allows state information to be stored on client machine
Http Servlet	Provide method to handle HTTP request and response
Http Session Event	Encapsulate session changed event
Http Session Binding Event	Indicates a Listener is bound to or unbound from a session

10.2.6. Cookie:

- A cookie is a small piece of information that is persisted between the multiple client requests.
- Cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.
- Cookies are text files stored on the client computer and they are kept for various information tracking purpose.

How Cookie works:

- By default, each request is considered as a new request.
- In cookies technique, we add cookie with response from the servlet. So, cookie is stored in the cache of the browser.
- After that if request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.

Types of Cookie:

There are 2 types of cookies in servlets.

- Non-persistent cookie
- Persistent cookie

Non-persistent cookie

- It is valid for single session only.
- It is removed each time when user closes the browser.

Persistent cookie

- It is valid for multiple session.
- It is not removed each time when user closes the browser. It is removed only if user logout or sign-out.

Advantage of Cookies:

- Simplest technique of maintaining the state.
- Cookies are maintained at client side.

Disadvantage of Cookies:

- It will not work if cookie is disabled from the browser.
- Only textual information can be set in Cookie object.

10.3 HTTP Request Handling, Session Management:**1) HTTP Request Handling:**

- The HttpServlet class provides specialized methods that handle the various types of HTTP requests.
- These methods are doDelete(), doGet(), doHead(), doOptions(), doPost(), doPut(), and doTrace().
- The GET and POST requests are commonly used when handling form input.

A) Handling HTTP GET Requests:

- Here we will develop a servlet that handles an HTTP GET request.
- The servlet is invoked when a form on a web page is submitted.

- The example contains two files. A web page is defined in ColorGet.htm, and a servlet is defined in ColorGetServlet.java.
- The HTML source code for ColorGet.htm is shown in the following listing.
- It defines a form that contains a select element and a submit button.

```
<html>
<body>
<center>
<formname="Form1"action="http://localhost:8080/servlets
examples/servlet/ColorGetServlet">
<B>Color:</B>
<select    name="color"    size="1">    <option    value="Red">Red</option><option
value="Green">Green</option> <option value="Blue">Blue</option>
</select>
<br><br>
<input type=submit value="Submit">
</form>
</body>
</html>
```

- The source code for ColorGetServlet.java is shown in the following listing.
- It uses the `getParameter( )` method of `HttpServletRequest` to obtain the selection that was made by the user.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorGetServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        String color = request.getParameter("color");
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<B>The selected color is: ");
        pw.println(color);
        pw.close();
    }
}
```

- After completing these steps, the browser will display the response that is dynamically generated by the servlet.
- One other point: Parameters for an HTTP GET request are included as part of the URL that is sent to the web server.
- Assume that the user selects the red option and submits the form.
- The URL sent from the browser to the server is :
`http://localhost:8080/servlets-examples/servlet/ColorGetServlet?color=Red`

B) Handling HTTP POST Requests:

- Here we will develop a servlet that handles an HTTP POST request.
- The servlet is invoked when a form on a web page is submitted.
- The example contains two files. A web page is defined in ColorPost.htm, and a servlet is defined in ColorPostServlet.java.
- The HTML source code for ColorPost.htm is shown below.
- It defines a form that contain a select element and a submit button.

```
<html>
<body>
<center>
<formname="Form1"method="post"action=
"http://localhost:8080/servletexamples/servlet/ColorPostServlet">
<B>Color:</B>
<select name="color" size="1">
<option value="Red">Red</option>
<option value="Green">Green</option>
<option value="Blue">Blue</option>
</select>
<br><br>
<input type="submit" value="Submit"> </form>
</body>
</html>
```

- The source code for ColorPostServlet.java is shown below.
- It uses the `getParameter()` method of `HttpServletRequest` to obtain the selection that was made by the user. A response is then formulated.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorPostServlet extends HttpServlet
{
    public void doPost(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException
    {
        String color = request.getParameter("color");
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<B>The selected color is: ");
        pw.println(color);
        pw.close();
    }
}
```

- Parameters for an HTTP POST request are not included as part of the URL that is sent to the web server.
- In this example, the URL sent from the browser to the server is `http://localhost:8080/servlets-examples/servlet/ColorPostServlet`.

2) Session Tracking:

- Session tracking is the mechanism to save state information so that information can be collected from several interactions between a browser and a server. Sessions provide such a mechanism.
- **Session Tracking** is a way to maintain state (data) of a user.
- It is also known as **session management** in servlet.
- Http protocol is a stateless so we need to maintain state using session tracking techniques.
- Each time user requests to the server, server treats the request as the new request.
- So, we need to maintain the state of an user to recognize to particular user.

❖ **Why use Session Tracking?**

- To recognize the user It is used to recognize the particular user.

❖ **Session Tracking Techniques:**

There are four techniques used in Session tracking:

- Cookies
- Hidden Form Field
- URL Rewriting
- HttpSession

- ✓ A session can be created via the getSession() method of HttpServletRequest.
 - The getSession() method gets the current session.
 - A new session is created if one does not already exist.
- ✓ The getAttribute() method is called to obtain the Date object that encapsulates the date and time when this page was last accessed.
- ✓ The setAttribute() method is called to bind the name “date” to this object.

The following servlet illustrates how to use session state

```
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DateServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {
        // Get the HttpSession object.

        HttpSession hs = request.getSession(true);

        //Get writer.
        response.setContentType("text/html"); PrintWriterpw=response.getWriter(); pw.print("<B>");

        //Display date/time of last access.
        Date date = (Date)hs.getAttribute("date"); if(date != null) {
        pw.print("Last access: " + date + "<br>"); }
        Display current date/time. date = new Date(); hs.setAttribute("date", date); pw.println("Current
        date: " + date);
    }
}
```


10.4 JSP Life Cycle, Writing JSP Pages:

- Java Server Pages (JSP) is a technology for developing Webpages that supports dynamic content.
- JSP technology is used to create web application just like Servlet technology.
- It can be thought of as an extension to Servlet because it provides more functionality than servlet such as expression language, JSTL, etc.
- This helps developers insert java code in HTML pages by making use of special JSP tags, most of which start with <% and end with %>.
- Web developers write JSPs as text files that combine HTML or XHTML code, XML elements, and embedded JSP actions and commands.
- Using JSP, we can collect input from users through Webpage forms, present records from a database or another source, and create Webpages dynamically.

❖ Why Use JSP?

- **Java Server Pages** often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But JSP offers several advantages in comparison with the CGI.
- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having a separate CGI file.
- JSP are always compiled before it's processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- Java Server Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.
- Finally, JSP is an integral part of J2EE, a complete platform for enterprise class applications. This means that JSP can play a part in the simplest applications to the most complex and demanding.

❖ Advantages of JSP:

Following is the list of other advantages of using JSP over other technologies:

- **vs. Active Server Pages (ASP):** The advantages of JSP are twofold. First, the dynamic part is written in Java, not Visual Basic or other MS specific language, so it is more powerful and easier to use. Second, it is portable to other operating systems and non-Microsoft Web servers.
- **vs. Pure Servlets:** It is more convenient to write (and to modify!) regular HTML than to have plenty of `println` statements that generate the HTML.
- **vs. Server-Side Includes (SSI):** SSI is really only intended for simple inclusions, not for "real" programs that use form data, make database connections, and the like.
- **vs. JavaScript:** JavaScript can generate HTML dynamically on the client but can hardly interact with the web server to perform complex tasks like database access and image processing etc.
- **vs. Static HTML:** Regular HTML, of course, cannot contain dynamic information.

❖ JSP Life Cycle

- A JSP life cycle can be defined as the entire process from its creation till the destruction which is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet.

The following are the paths followed by a JSP

- Compilation
- Initialization
- Execution
- Cleanup

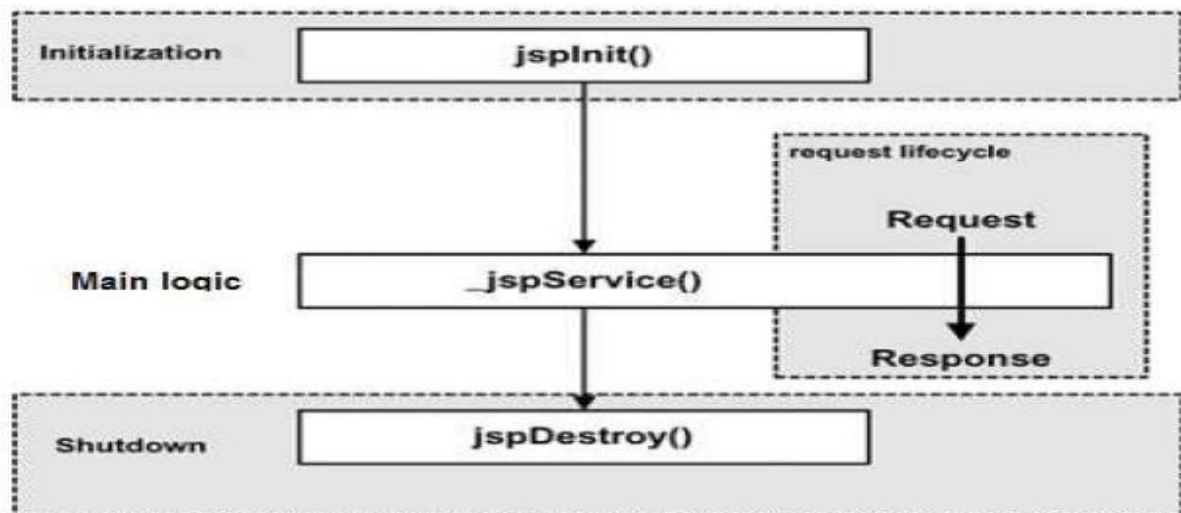


Fig: JSP Life Cycle

JSP Compilation:

- When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page.
- The compilation process involves three steps:
 1. Parsing the JSP.
 2. Turning the JSP into a servlet.
 3. Compiling the servlet.

JSP Initialization:

- When a container loads a JSP it invokes the `jspInit()` method before servicing any requests.
- If we need to perform JSP-specific initialization, override the `jspInit()` method:

```
public void jspInit()
{
    Initialization code...
}
```

- Typically, initialization is performed only once and as with the servlet `init` method, we generally initialize database connections, open files, and create lookup tables in the `jspInit` method.

JSP Execution:

- This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.
- Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the `_jspService()` method in the JSP.

The `_jspService()` method takes an `HttpServletRequest` and an `HttpServletResponse` as its parameters as follows:

```
void _jspService(HttpServletRequest request,  
HttpServletResponse response)  
{  
    Service handling code...  
}
```

- The `_jspService()` method of a JSP is invoked once per a request and is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods ie. GET, POST, DELETE etc.

JSP Cleanup:

- The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container.
- The `jspDestroy()` method is the JSP equivalent of the `destroy` method for servlets.
- We need to override `jspDestroy` to perform any cleanup, such as releasing database connections or closing open files.

The `jspDestroy()` method has the following form:

```
public void jspDestroy()  
{  
    Your cleanup code goes here.  
}
```

❖ JSP Syntax:

The JSP elements are:

- Declaration
- Expression
- Scriptlets
- Directives

1) JSP Element Declaration:

- The `declarations` tag allows us to define variables and methods.
- Variable declarations are compiled as instance variables for a class that corresponds to the JSP page.

Declarations are defined with the syntax

```
<%!
```

```
Declarations
```

```
%>
```

```
Example,  
<%!  
private int count = 0;  
private void incrementCount()  
{  
count++;  
}  
%>
```

2) JSP Expression:

- We can access variables defined in declarations with expression.
- Expressions are embedded directly into the HTML.
- The Web browser will display the value of the expression in place of the tag.

The syntax to embed an expression is as follows:

```
<%=  
Expression  
%>
```

Example:

The value of count is <%= count %>

3) JSP Scriptlet:

- Blocks of Java code can be embedded in a scriptlet.
- The output of HTML within a scriptlet, then this is done using out.println(), which is used in the same manner as System.out.println().

The syntax for a scriptlet is as follows:

```
<%  
Java code  
%>
```

Example:

```
<%  
out.println("The counter's value is " + count + "<br />"); incrementCount();  
%>
```

4) JSP Directives:

- Directives are the ones that instruct the compiler how to process a JSP program.
- Examples include the definition of our own tags, including the source code of other files, and importing packages.

The syntax for directives is as follows:

```
<%@  
page import="java.util.*,java.sql.*"  
%>
```

10.5 Introduction to JSP Tag Library:

- Java Server Pages Standard Tag Library (JSTL) is a collection of useful JSP tags which encapsulates core functionality common to many JSP applications.

The JSTL tags can be classified, according to their functions, into following JSTL tag library groups that can be used when creating a JSP page:

- ✓ Core Tags
- ✓ Formatting tags
- ✓ SQL tags
- ✓ XML tags
- ✓ JSTL Functions

1) Core Tags:

- The core group of tags is the most frequently used JSTL tags.

Following is the syntax to include JSTL Core library in our JSP:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

Tag	Description
<u><c:out ></u>	Like <%= ... >, but for expressions.
<u><c:set ></u>	Sets the result of an expression evaluation in a 'scope'
<u><c:remove ></u>	Removes a scoped variable (from a particular scope, if specified).
<u><c:catch></u>	Catches any Throwable that occurs in its body and optionally exposes it.
<u><c:if></u>	Simple conditional tag which evaluates its body if the supplied condition is true.
<u><c:choose></u>	Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by <when> and <otherwise>
<u><c:when></u>	Subtag of <choose> that includes its body if its condition evaluates to 'true'.
<u><c:otherwise ></u>	Subtag of <choose> that follows <when> tags and runs only if all of the prior conditions evaluated to 'false'.
<u><c:import></u>	Retrieves an absolute or relative URL and exposes its contents to either the page, a String in 'var', or a Reader in 'varReader'.
<u><c:forEach ></u>	The basic iteration tag, accepting many different collection types and supporting subsetting and other functionality .
<u><c:forTokens></u>	Iterates over tokens, separated by the supplied delimiters.
<u><c:param></u>	Adds a parameter to a containing 'import' tag's URL.
<u><c:redirect ></u>	Redirects to a new URL.
<u><c:url></u>	Creates a URL with optional query parameters

2) Formatting tags:

- The JSTL formatting tags are used to format and display text, the date, the time, and numbers for internationalized Web sites.

Following is the syntax to include formatting library in our JSP:

```
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
```


Tag	Description
<u><fmt:formatNumber></u>	To render numerical value with specific precision or format.
<u><fmt:parseNumber></u>	Parses the string representation of a number, currency, or percentage.
<u><fmt:formatDate></u>	Formats a date and/or time using the supplied styles and pattern
<u><fmt:parseDate></u>	Parses the string representation of a date and/or time
<u><fmt:bundle></u>	Loads a resource bundle to be used by its tag body.
<u><fmt:setLocale></u>	Stores the given locale in the locale configuration variable.
<u><fmt:setBundle></u>	Loads a resource bundle and stores it in the named scoped variable or the bundle configuration variable.
<u><fmt:timeZone></u>	Specifies the time zone for any time formatting or parsing actions nested in its body.
<u><fmt:setTimeZone></u>	Stores the given time zone in the time zone configuration variable
<u><fmt:message></u>	To display an internationalized message.
<u><fmt:requestEncoding></u>	Sets the request character encoding

3) SQL tags

- The JSTL SQL tag library provides tags for interacting with relational databases (RDBMSs) such as Oracle, MySQL, or Microsoft SQL Server.

Following is the syntax to include JSTL SQL library in our JSP:

```
<%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>
```

Tag	Description
<u><sql:setDataSource></u>	Creates a simple DataSource suitable only for prototyping
<u><sql:query></u>	Executes the SQL query defined in its body or through the sql attribute.
<u><sql:update></u>	Executes the SQL update defined in its body or through the sql attribute.
<u><sql:param></u>	Sets a parameter in an SQL statement to the specified value.
<u><sql:dateParam></u>	Sets a parameter in an SQL statement to the specified java.util.Date value.
<u><sql:transaction ></u>	Provides nested database action elements with a shared Connection, set up to execute all statements as one transaction.

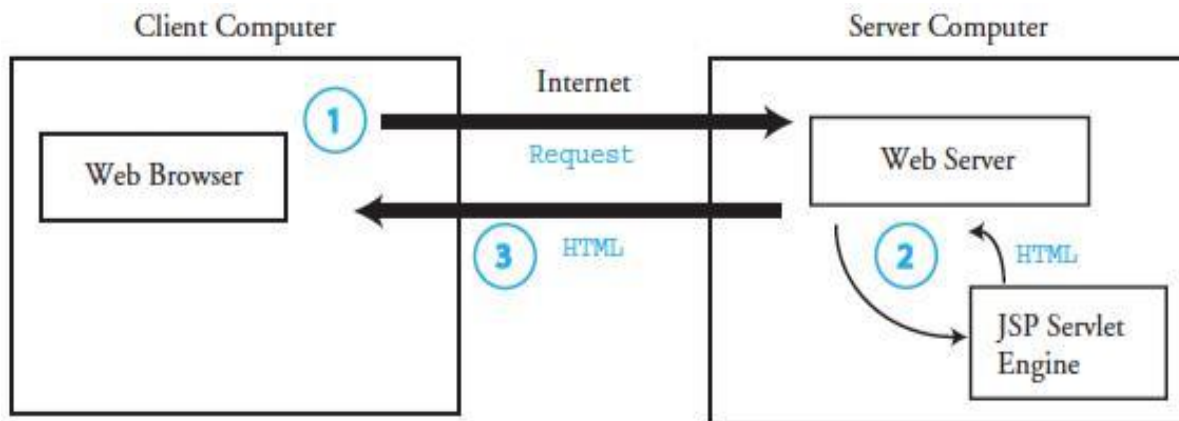
4) XML tags:

- The JSTL XML tags provide a JSP-centric way of creating and manipulating XML documents. Following is the syntax to include JSTL XML library in our JSP.
- The JSTL XML tag library has custom tags for interacting with XML data.
- This includes parsing XML, transforming XML data, and flow control based on XPath expressions.

```
<%@ taglib prefix="x" uri="http://java.sun.com/jsp/jstl/xml" %>
```

Tag	Description
<u><x.out></u>	Like <%= ... >, but for XPath expressions.
<u><x.parse></u>	Use to parse XML data specified either via an attribute or in the tag body.
<u><x.set ></u>	Sets a variable to the value of an XPath expression.
<u><x.if ></u>	Evaluates a test XPath expression and if it is true, it processes its body. If the test condition is false, the body is ignored.
<u><x.forEach></u>	To loop over nodes in an XML document.
<u><x.choose></u>	Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by <when> and <otherwise>
<u><x.when ></u>	Subtag of <choose> that includes its body if its expression evaluates to 'true'
<u><x.otherwise ></u>	Subtag of <choose> that follows <when> tags and runs only if all of the prior conditions evaluated to 'false'
<u><x.transform ></u>	Applies an XSL transformation on a XML document
<u><x.param ></u>	Use along with the transform tag to set a parameter in the XSLT stylesheet

❖ Running a JSP:



- ① The client's Web browser sends a request to the server for a Web page that contains JSP code.
- ② The JSP servlet engine dynamically compiles the JSP source code into a Java servlet if a current, compiled servlet doesn't exist. The servlet runs and outputs HTML that is returned to the Web server.
- ③ The Web server sends the servlet's HTML to the client's Web browser to be displayed.

Fig: Running a Java Server Page (JSP) Program

Simple JSP example:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>JSP simple program</title>
</head>
<body>
<%= "Hello World!" %>
</body>
</html>
```

Output:

Hello world!

Important Questions for Exams:

- 1) Define Servlet? Explain life cycle of servlet?
- 2) Explain cookie and session tracking in java?
- 3) Define JSP? Explain the JSP tags with the simple JSP Example?
- 4) How does the HTTP handles requests? explain in detail the differences between GET and POST request?
- 5) Write down the jsp code to create web page which has two labels name username and password and response to user input as well?
- 6) Differences between servlet and JSP? Explain the life cycle of JSP?
- 7) What is servlet API and explain how does an applet runs in the machine?
- 8) Explain JSP syntax in detail? Create a simple page with two labels and text fields along with the submit button.